AIAC ASSIGNMENT – 3.1

Name : VALLALA VYSHNAVI Batch : 32 HNO : 2303A52429

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate

a Python function that checks whether a given number is a palindrome.

#prompt

#code

```python
def is_palindrome_number(num):
    """
    Check if the given integer is a palindrome number.

    A palindrome number is a number that remains the same when its digits are reversed.

    Parameters:
    num (int): The integer to be checked.

    Returns:
    bool: True if the number is a palindrome, False otherwise.
    """
    # Convert the integer to string
    str_num = str(num)
    # Check if the string is equal to its reverse
    return str_num == str_num[::-1]

# Dynamic input from the user
if __name__ == "__main__":

    try:
        user_input = int(input("Enter an integer to check if it is a palindrome: "))
        if is_palindrome_number(user_input):
            print(f"{user_input} is a palindrome number.")
        else:
            print(f"{user_input} is not a palindrome number.")
    except ValueError:
        print("Please enter a valid integer.")
```

#output

```
PS C:\Users\Admin> & C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/jan 13th.py"
Enter an integer to check if it is a palindrome: 22
22 is a palindrome number.
```

 # Explanation

```
1. Defines a function with docstrings to check whether a number is a palindrome.
2. Converts the integer input into a string for easy reversal.
3. Compares the original string with its reversed version.
4. Accepts dynamic input from the user at runtime.
5. Handles invalid input using error handling to avoid program crashes.
'''
```

Question 2: One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a

given number.

Example:

Input: 5 → Output: 120

#prompt

```
#num=5
# return factorial of a number
```

#code

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
print(factorial(5))
```

#output

```
PS C:\Users\Admin> & C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/jan 13th.py"
120
```

 # Explanation

- Providing one input-output example helps the AI clearly understand the expected behavior.This improves correctness and clarity compared to zero-shot prompting.

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples

to guide the AI in generating a Python function to check whether a

given number is an Armstrong number.

Examples:

• Input: 153 → Output: Armstrong Number

• Input: 370 → Output: Armstrong Number

• **Input: 123 → Output: Not an Armstrong Number**

#prompt

```
'''write a python program to check whether the given number is armstorng number or not
153=armstorng number
370=armstorng number
12=not armstorng number
'''
```

#code

```
def is_armstrong_number(n):
    """Check if a number is an Armstrong number.
    Args:
        n (int): The number to check.
    Returns:
        bool: True if n is an Armstrong number, False otherwise.    """
    num_str = str(n)
    num_digits = len(num_str)
    sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
    return sum_of_powers == n
# Example usage
number = int(input("Enter a number: "))
if is_armstrong_number(number):
    print(f"{number} is an Armstrong number.")
else:
    print(f"{number} is not an Armstrong number.")
```

#output

```
PS C:\Users\Admin> & C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/jan 13th.py"
Enter a number: 153
```

 # Explanation

- AI helps generate accurate, efficient Armstrong number code quickly.
- AI simplifies logic and reduces coding errors for Armstrong programs.

Question 4: Context-Managed Prompting (Optimized Number

Classification)

Design a context-managed prompt with clear instructions and

constraints to generate an optimized Python program that classifies a

number as prime, composite, or neither

#prompt

#code

```
def classify_number(n):
    if n <= 1:
        return "neither"
    elif n == 2:
        return "prime"
    elif n % 2 == 0:
        return "composite"

    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return "composite"

    return "prime"
# Get user input
try:
    number = int(input("Enter a number: "))
    classification = classify_number(number)
    print(f"The number {number} is classified as: {classification}")
except ValueError:
    print("Please enter a valid integer.")
```

#output

```
Python/Python312/python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/jan 13th.py"
Enter a number: 5
The number 5 is classified as: prime
PS C:\Users\Admin> & C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/jan 13th.py"
Enter a number: 1
The number 1 is classified as: neither
PS C:\Users\Admin> & C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/jan 13th.py"
Enter a number: 8
The number 8 is classified as: composite
```

 # Explanation

Question 5: Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to

generate a Python function that checks whether a given number is a

perfect number.

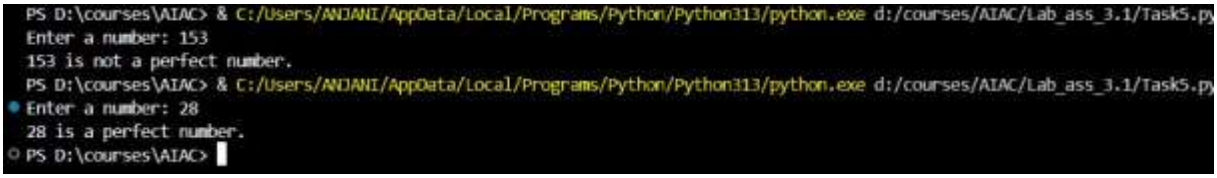
#prompt

#code

```python
def is_perfect_number(n):
    if n <= 1:
        return False
    divisors = [i for i in range(1, n) if n % i == 0]
    return sum(divisors) == n

number = int(input("Enter a number: "))
if is_perfect_number(number):
    print(f"{number} is a perfect number.")
else:
    print(f"{number} is not a perfect number.")
```

#output



 # Explanation

- This zero-shot prompt checks basic AI understanding . Logic may need optimization after testing.
- AI quickly generates clear code to identify prime, composite, or neither numbers.

AI improves accuracy and saves time when writing prime number logic.

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper

input validation.

Examples:

- **Input: 8 → Output: Even**

• Input: 15 → Output: Odd

• Input: 0 → Output: Even

#prompt

```
'''return as even or odd number
8-even
14-odd
0-even'''
```

#code

```
def even_odd(num):
    if num % 2 == 0:
        return "even"
    else:
        return "odd"
number=int(input("Enter a number: "))
result=even_odd(number)
print(f"{number}-{result}")
```

#output

```
Enter a number: 8
8-even
PS C:\Users\Admin> & C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/jan 13th.py"
Enter a number: 15
15-odd
PS C:\Users\Admin> & C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/jan 13th.py"
Enter a number: 0
0-even
```

 # Explanation

- AI helps generate code quickly to identify patterns like even or odd numbers and explains the logic clearly.
- AI improves code quality by suggesting better variable names, structure, debugging, and optimization.